

Lab 6: CRUD on one table and more...

This lab session is dedicated to allowing the administrator - that can be hardcoded in your application (no need to put the administrator in the table “users”, especially if you do not have one) – to use CRUD on one table (I will take the “projects” table in this lab. We consider you did everything until the end of lab 5 because we will continue with the “projects and skills” database in this session. To try and test some code, you can use the data I provide for the labs, but in the end, I recall that the database of your project **MUST** use **YOUR (OWN) DATA!**

1) Making each project clickable

Follow the instructions and code given in lecture 18 to be able to make a project clickable and create a new handlebar to show details about this project.

Try your code and click on a project to get detailed information about it.

2) CRUD operations on the projects

2.1) Modifying “projects.handlebars”

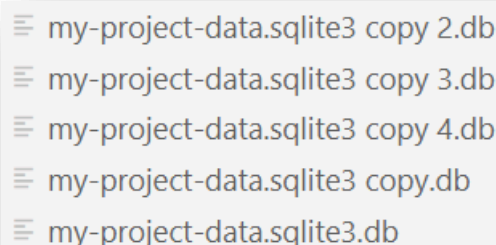
To allow the CUD operations, we must ensure that the user is an admin. Follow the instructions and code given in lecture 18 to be able to do the CUD operations on the projects.

2.2) Delete a project

Follow the instructions and code given in lecture 18 to be able to delete a project.

Try this code, **BE CAREFUL**, if it works, it will delete some of your rows in the database. I suggest that you make a backup copy of your “clean” database file **BEFORE** running the delete code. Having a backup is a good idea to have a version of your data with all the rows.

Personally, I did multiple copies (copy and paste) of the database file, so I can delete many rows and still be able to delete my incomplete database file and rename one of the copy with the name of the original to have my data back. :)



```
my-project-data.sqlite3 copy 2.db
my-project-data.sqlite3 copy 3.db
my-project-data.sqlite3 copy 4.db
my-project-data.sqlite3 copy.db
my-project-data.sqlite3.db
```

2.3) Create a new project

Follow the instructions and code given in lecture 18 to be able to create a new project.

Try your code and create a new project.

2.3) Modify an existing project

If we take two minutes to think about it, modifying a project needs exactly the same form as creating a new one, except that we will pre-fill all the fields.

How to do that? We will send a model containing the current information on the project to the same template “project-form.handlebars” and pre-fill the form fields with them! Of course, we need to change the destination of the form according to the creation of a new project or the modification of an existing one, but we will use the same template for both creation and modification.

Follow the instructions and code given in lecture 18 to be able to create a new project.

Try your code and modify a project.

3) Error and default routes

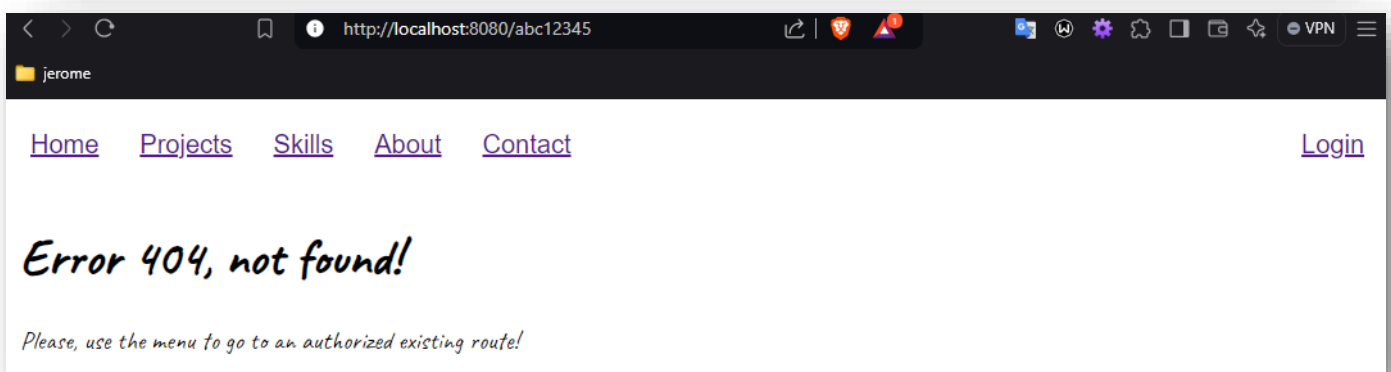
3.1) Default 404 (not found page)

If you want to deal with a non-existing route (error 404), you can add a middleware, it should be placed after your routes, just before the “app.listen(...)” function:

```
//-----  
// 404 NOT FOUND  
//-----  
app.use(function(req,res){ // the default 404 error  
  res.status(404).render('404.handlebars');  
})  
  
//-----  
// LISTEN  
//-----  
app.listen(port, function () { // listen to the port
```

Because it is a route leading to a 404.handlebars template, you must define one:

```
views > 404.handlebars > ...  
1      <div id="principal">  
2          <h1>Error 404, not found!</h1>  
3          <div class="mytextcards">  
4              Please, use the menu to go to an authorized existing route!  
5          </div>  
6      </div>  
7
```



This “404” route will be sent if the route asked by the user does not exist (for example abc12345 in the picture).

3.2) Other error (status 500 in http)

If you want to deal with any other error, you need a status 500 (error) checking, the code can be put just after the 404 error page; so before “app.listen(...)”:

```
//-----  
// 404 NOT FOUND  
//-----  
app.use(function(req,res){ // the default 404 error  
|   res.status(404).render('404.handlebars');  
| })  
  
//-----  
// 500 ERROR  
//-----  
app.use(function(err,req,res,next){  
|   console.error(err.stack);  
|   res.status(500).render('500');  
| });
```

```
views > 500.handlebars > ...  
1 | <div id="principal">  
2 |   <h1>Error 500, unexpected unknown error!</h1>  
3 |   <div class="mytextcards">  
4 |     Please, use the menu to go to an authorized existing route!  
5 |   </div>  
6 | </div>
```

These two codes come from (Stackoverflow, 2016,
<https://stackoverflow.com/questions/37357687/express-handlebars-page-is-routing-to-404>).

Please, remember that:

Web Dev is Fun

:)